TALLER RESUELTO

Pilas, colas y listas enlazadas

Parte 1. Reconocer la estructura adecuada

Situación

Estructura

Justificación

Deshacer la última acción de un editor

Pila

Sigue LIFO: la última acción guardada es la primera que se retira.

Atender personas por orden de llegada

Cola

Sigue FIFO: quien llega primero se atiende primero.

Volver a la primera canción al terminar la última

Lista circular

El último nodo apunta al primero y permite continuar el recorrido.

Recorrer registros en un sentido y agregar al comienzo

Lista simple

Cada nodo apunta al siguiente; insertar al principio requiere actualizar cabeza.

Pasar a la canción siguiente y anterior

Lista doble

Cada nodo mantiene referencias al anterior y al siguiente.

Parte 2. Pilas (LIFO)

2.1. Seguimiento de las operaciones

El tope se muestra a la izquierda de cada cadena. push agrega en el tope; pop retira y devuelve el elemento superior; peek consulta el tope sin retirarlo.

Operación

Estado después de la operación

1. push(4)

```
TOPE → [4]
```

2. push(9)

```
TOPE → [9] → [4]
```

3. push(2)

```
TOPE → [2] → [9] → [4]
```

4. pop()

```
Devuelve 2. Queda: TOPE → [9] → [4]
```

5. push(7)

```
TOPE → [7] → [9] → [4]
```

6. peek()

```
Devuelve 7. La pila no cambia: TOPE → [7] → [9] → [4]
```

7. pop()

```
Devuelve 7. Queda: TOPE → [9] → [4]
```

8. pop()

```
Devuelve 9. Queda: TOPE → [4]
```

Resultado final: queda el elemento 4 y es el tope de la pila.

2.2. Casos límite

```
pop() en una pila vacía: no hay elemento que retirar. Se devuelve un error o un valor especial como NULO, sin
```

modificar la pila.

```
peek() en una pila vacía: indica que no existe un tope; puede devolver NULO o un mensaje de error.
```

Pseudocódigo de deshacer(pila):

```
Funcion deshacer(pila)
```

Si pila está vacía Entonces

Escribir "No hay acciones para deshacer"

```
Retornar NULO
```

FinSi

accion <- pop(pila)

Retornar accion

FinFuncion

2.3. Aplicación: deshacer acciones

Las acciones se realizaron en este orden: escribir, borrar y pegar.

Antes del primer deshacer:

```
TOPE → [pegar] → [borrar] → [escribir]
```

Primer deshacer: retira «pegar».

Antes del segundo deshacer:

```
TOPE → [borrar] → [escribir]
```

Segundo deshacer: retira «borrar».

```
Pila final: TOPE → [escribir]
```

La primera acción que se deshace es «pegar» y la segunda es «borrar». La pila representa este comportamiento porque aplica LIFO: lo último que entra es lo primero que sale.

Parte 3. Colas (FIFO)

3.1. Simulación de la fila de atención

En cada diagrama, el frente está a la izquierda y el final a la derecha.

Operación

Estado / resultado

1. encolar("Ana")

```
Frente → [Ana] ← Final
```

2. encolar("Luis")

```
Frente → [Ana] → [Luis] ← Final
```

3. encolar("Sol")

```
Frente → [Ana] → [Luis] → [Sol] ← Final
```

4. desencolar()

```
Devuelve Ana. Queda: Frente → [Luis] → [Sol] ← Final
```

5. encolar("Mia")

```
Frente → [Luis] → [Sol] → [Mia] ← Final
```

6. desencolar()

```
Devuelve Luis. Queda: Frente → [Sol] → [Mia] ← Final
```

7. encolar("Teo")

```
Frente → [Sol] → [Mia] → [Teo] ← Final
```

Resultado final: esperan Sol, Mia y Teo, en ese orden. Sol está al frente y Teo al final.

3.2. Operaciones y casos límite

Pseudocódigo de atender(cola):

```
Funcion atender(cola)
```

Si cola está vacía Entonces

Escribir "No hay personas esperando"

```
Retornar NULO
```

FinSi

persona <- desencolar(cola)

Retornar persona

FinFuncion

```
Desencolar una cola vacía: se informa que no hay personas esperando y se devuelve NULO o un error. La cola
```

permanece sin cambios.

Agregar al frente y retirar desde el final: con esas operaciones se invierte el orden de llegada, produciendo un comportamiento tipo LIFO. Para una cola FIFO se inserta al final y se retira desde el frente.

Parte 4. Listas enlazadas simples

4.1. Construcción y modificación

```
Cada nodo contiene un dato y una referencia siguiente. El último apunta a NULO.
```

Operación

Lista resultante

1. Insertar 18 al final

```
cabeza → [18] → NULO
```

2. Insertar 7 al comienzo

```
cabeza → [7] → [18] → NULO
```

3. Insertar 24 al final

```
cabeza → [7] → [18] → [24] → NULO
```

4. Insertar 13 después de 7

```
cabeza → [7] → [13] → [18] → [24] → NULO
```

5. Eliminar 18

```
cabeza → [7] → [13] → [24] → NULO
```

Al eliminar 18, se cambia la referencia siguiente del nodo 13 para que apunte a 24. Así se conserva el resto de la cadena.

4.2. Diseño de operaciones

a) Recorrer e imprimir:

```
Procedimiento imprimirLista(cabeza)
actual <- cabeza
Mientras actual != NULO Hacer
```
Escribir actual.dato
```
actual <- actual.siguiente
```
FinMientras

FinProcedimiento

b) Buscar un valor:
```
Funcion buscar(cabeza, valor)
actual <- cabeza
Mientras actual != NULO Hacer
```
Si actual.dato = valor Entonces

Retornar VERDADERO

FinSi
```
actual <- actual.siguiente
```
FinMientras

Retornar FALSO

FinFuncion

c) Insertar al comienzo:
```
Procedimiento insertarComienzo(Referencia cabeza, valor)
```
nuevo <- crearNodo(valor)

nuevo.siguiente <- cabeza
```
cabeza <- nuevo
```
FinProcedimiento

4.3. Comprobación de los bordes
```
Lista vacía: antes, cabeza → NULO. Al insertar 7, queda cabeza → [7] → NULO. El nuevo nodo pasa a ser la
```
cabeza.
```
Lista con nodos: antes, cabeza → [18] → [24] → NULO. Al insertar 7, queda cabeza → [7] → [18] → [24] → NULO.
```
Eliminar el primer nodo: si hay varios nodos, la cabeza se mueve al segundo nodo. Si solo había uno, la cabeza
```
se actualiza a NULO.
Antes: cabeza → [7] → [13] → [24] → NULO
Después: cabeza → [13] → [24] → NULO
```

Parte 5. Listas circulares

5.1. Reproducción de un recorrido circular
```
Lista inicial: [P1] → [P2] → [P3]
```
↑_____|

El último nodo, P3, vuelve a apuntar a P1.

Se supone que todos los procesos están activos al comienzo y que P2 se retira justo después de completar su segundo turno (turno 5).

Turno

Atendido

Estado al terminar

Siguiente

1

P1

Continúa activo

P2

2

P2

Continúa activo

P3

3

P3

Continúa activo

P1

4

P1

Continúa activo

P2

5

P2

Termina y se retira

P3

6

P3

Continúa activo

P1

Turno

Atendido

Estado al terminar

Siguiente

7

P1

Continúa activo

P3

Lista después de retirar P2:

```
[P1] → [P3]
↑_____|
```

Se actualiza P1.siguiente para que apunte a P3. El enlace de P3 vuelve a P1, por lo que la lista sigue siendo circular.

```
Condición de recorrido: guardar la cabeza y detenerse cuando el recorrido regrese a ella. No se espera NULO
```

porque no existe un final nulo en una lista circular.

```
Si cabeza != NULO Entonces
actual <- cabeza
```

Repetir

Procesar(actual.dato)

```
actual <- actual.siguiente
```

Hasta Que actual = cabeza

FinSi

5.2. Casos especiales

Lista vacía: no hay enlaces que actualizar; una eliminación debe informar que la lista está vacía.

```
Un solo nodo: el nodo se apunta a sí mismo. Al eliminarlo, cabeza pasa a NULO.
```

Un nodo, antes: [A] I

```
Después de eliminar: cabeza → NULO
```

Eliminar el primero: la cabeza pasa al segundo nodo y el último se enlaza con la nueva cabeza. Si solo hay uno, la lista queda vacía.

Eliminar el último: se busca el nodo anterior al último y se hace que su siguiente apunte a la cabeza. En una lista circular simple, normalmente hay que recorrerla para encontrar ese anterior.

Dos o más nodos, antes:

```
cabeza → [A] → [B] → [C] II
↑_____________________|
```

Después de eliminar C:

```
cabeza → [A] → [B] II
↑_________________|
```

Parte 6. Listas doblemente enlazadas

6.1. Recorrido en ambos sentidos

```
Cada nodo guarda el dato y dos referencias: anterior y siguiente. El anterior del primer nodo es NULO y el siguiente del último también es NULO.
NULO ← [Lunes] I [Martes] I [Miércoles] I [Jueves] → NULO
Recorrido de comienzo a final: Lunes → Martes → Miércoles → Jueves.
Recorrido de final a comienzo: Jueves → Miércoles → Martes → Lunes.
```

6.2. Inserta y elimina

1. Insertar Domingo antes de Lunes:

```
NULO ← [Domingo] I [Lunes] I [Martes] I [Miércoles] I [Jueves] → NULO
```

Domingo.siguiente apunta a Lunes y Lunes.anterior apunta a Domingo. Domingo se convierte en la nueva cabeza.

2. Insertar Viernes después de Jueves:

```
NULO ← [Domingo] I [Lunes] I [Martes] I [Miércoles] I [Jueves] I [Viernes] → NULO
Se conectan Jueves.siguiente con Viernes y Viernes.anterior con Jueves. Viernes.siguiente queda en NULO.
```

3. Eliminar Miércoles:

Antes: [Martes] I [Miércoles] I [Jueves]

Cambios: Martes.siguiente = Jueves

Jueves.anterior = Martes

Resultado:

```
NULO ← [Domingo] I [Lunes] I [Martes] I [Jueves] I [Viernes] → NULO
```

6.3. Comparación de los enlaces

Una lista doble permite recorrer los elementos hacia adelante y hacia atrás, como al pasar de Jueves a Lunes en el ejercicio anterior. También facilita desconectar un nodo conocido sin buscar su anterior. Su costo es que cada nodo almacena una referencia adicional y las inserciones y eliminaciones requieren actualizar más enlaces. Por eso, utiliza más memoria que una lista simple.

Parte 7. Análisis y elección

Estructura y

representación

Operación

Compleji

dad

Justificación breve

Pila enlazada con referencia

al tope

Insertar en el tope

O(1)

Crear nodo, conectarlo al tope y actualizar la referencia.

Pila enlazada con referencia

al tope

Retirar del tope

O(1)

Retirar el nodo superior y mover la referencia al siguiente.

Cola enlazada con frente y

final

Encolar al final

O(1)

La referencia al último permite insertar directamente.

Cola enlazada con frente y

final

Desencolar del

frente

O(1)

Se mueve el frente al siguiente; si queda vacía, final

también pasa a NULO.

Lista simple sin referencia al

último

Insertar al comienzo

O(1)

Solo se actualiza la cabeza y el enlace del nuevo nodo.

Lista simple sin referencia al

último

Insertar al final

O(n)

Se recorre desde la cabeza hasta el último nodo.

Lista doble con referencia a

nodo conocido

Desconectar ese

nodo

O(1)

Se ajustan los enlaces de los vecinos sin buscar el nodo; se

asume que la referencia ya está disponible.

¿Por qué una cola con solo referencia al frente puede tardar más al encolar al

final?

Debe recorrer la lista desde el frente hasta encontrar el último nodo. Si hay n elementos, la operación puede

visitar los n nodos y tarda O(n). Con una referencia al último nodo, insertar al final tarda O(1).

¿Qué ventaja ofrece una lista circular?

Su último nodo apunta al primero, por lo que el recorrido puede continuar desde el comienzo sin reconstruir la lista. Es útil para reproducir canciones en bucle o asignar turnos repetidos.

¿Qué almacena de más una lista doble y qué permite?

Almacena una referencia al nodo anterior además de la referencia al siguiente. Esto permite recorrer en ambos sentidos y facilita ciertas inserciones y eliminaciones, a cambio de mayor uso de memoria y más enlaces que mantener.

Parte 8. Reflexión

La forma en que se conectan los nodos influye en la manera de organizar y retirar los datos de una estructura. Por ejemplo, la pila utiliza el orden LIFO, porque el último elemento que se agrega es el primero que sale. En cambio, la cola utiliza FIFO, ya que atiende primero al elemento que llegó antes. La lista simple permite recorrer los datos en un solo sentido, mientras que la lista circular conecta el último nodo con el primero para repetir el recorrido. Por su parte, la lista doble permite avanzar y retroceder gracias a sus dos enlaces. Cada estructura tiene características diferentes y elegir la adecuada depende de las operaciones que se necesiten realizar.